

TOHSENO

LOHSENO

A Protocol for Turning Intention into Living Software

GENESIS WHITEPAPER AND CONSTITUTIONAL SPECIFICATION

Version 0.2 --- July 26, 2026

Status: Foundational, pre-stable-release (factory 0.5.0)

Origin: JP Franetovic / Anky, Inc.

Give every idea a shot.

The factory is local. Identity is held. Memory is networked.

Most shots miss. The prototype is the payoff.

TOH / ROLL 01 / WRITTEN BEFORE THE FIRST STABLE RELEASE

A note to whoever finds this

TOHSENO began with a simple observation:

A person can carry a coherent piece of software in their mind long before they know how to build it, distribute it, maintain it, or explain it to anyone else.

For most of the history of computing, converting that intention into software required a project: a team, a roadmap, a repository, a product manager, infrastructure, accounts, capital, and months of sustained coordination. The cost of producing software was so high that only a small fraction of human intentions were allowed to become executable.

That assumption is breaking.

The cost of expressing software is collapsing. A sufficiently clear intention can increasingly become a functioning application in a single sustained act. But the systems around software have not caught up. Creation may become fast while everything that gives an application a life remains fragmented: ownership, evolution, testing, feedback, identity, distribution, reputation, provenance, monetization, and memory.

TOHSENO exists to close that circle.

It is not merely a generator. It is not a cloud IDE, a token launcher, a marketplace, or a wrapper around a coding agent. It is an open protocol and local-first factory for turning a coherent intention into an independent application, allowing that application to evolve through contact with reality, and optionally connecting it to a shared network without surrendering its sovereignty.

And it makes a second observation, quieter but just as load-bearing:

*Most shots miss.
That is not a tragedy to be engineered away.
It is the reason the factory must make taking another one cheap.*

A photographer does not mourn a frame that did not land. The frame goes on the contact sheet, it teaches the eye, and the roll continues. TOHSENO treats software the same way. An app idea is a Shot, not a forever bet. Every Shot lands in the builder's contact sheet as a working artifact they can use, change, keep, or move past. The prototype is the payoff. Volume reveals signal. Taste emerges through making.

This document records the system before its first stable release.

It is written so that a future builder, machine, node operator, historian, or stranger can reconstruct not only how TOHSENO worked, but what it believed software should become.

Abstract

TOHSENO introduces the **Shot** as a primitive for personal software.

A Shot is one coherent intention embodied as an independent application and carried forward through a continuous lineage. The first intent creates the Shot. Subsequent intents do not create new Shots; they create **Evolutions** of the same Shot.

A Shot has three canonical lifecycle states:

| EVOLVING -> PUBLISHED -> APP_STORE

EVOLVING means the Shot exists primarily within the builder's local environment. **PUBLISHED** means its source is available through the TOHSENO ecosystem so that other people can inspect, download, build, run, and test it. **APP_STORE** means the Shot has been released through Apple's App Store. These states are distribution milestones, not the end of development. A Shot may continue evolving indefinitely after publication or release.

The TOHSENO factory is open-source and local-first. A builder must be able to install TOHSENO, create a Shot, evolve it, verify it, publish its source, and ship it without a TOHSENO account, mobile app, wallet, blockchain transaction, token, or official server. Every generated application must remain independent and operable if TOHSENO disappears — *ejectable from birth*, a property true at creation time, not a later export feature.

Inside the factory, responsibility is divided by a strict boundary: artificial intelligence interprets meaning; deterministic code owns composition, locks, storage invariants, verification, and authority. The coding agent builds, but TOHSENO — deterministic, verifiable, unimpressed — is the only party allowed to say the result works.

Around that sovereign local factory, TOHSENO defines an optional continuity layer: portable builder identity; app-specific user identity; explicit cross-app consent; mobile signing and approvals; public builder records; structured feedback; sharpened next intents; discovery; node-backed availability; append-only provenance; and deployment-agnostic appcoin links.

The future TOHSENO mobile app is a premium client for this continuity layer. It is not mandatory and it is not the root dependency of generated applications. It is subject to a genesis invariant: **it must not be built by hand before the protocol is stable. It must be the first canonical Shot created by the first stable TOHSENO release.**

TOHSENO nodes are compatible services that preserve and serve public memory. Nodes index; builders own. The official node is only the first node, not the definition of the network. An append-only registry may anchor public claims on an existing blockchain; it records small signed facts and content hashes, and nothing else. Appcoin deployment remains agnostic: TOHSENO records the relationship between a Shot and its canonical appcoin, and does not deploy, mint, trade, price, or endorse the asset.

The result is a system in which:

| one intention becomes one Shot;
| one Shot can evolve for years;

the source remains owned by the builder;
identity remains held by the human;
the network remembers what the builder chooses to make public;
and the application can acquire users, history, reputation,
and economics without becoming dependent on a single company.

Contents

1	The name	6
2	The problem	6
3	The foundational primitive: the Shot	8
4	Intent and Evolution	10
5	Lifecycle	12
6	The local factory	13
7	How this document speaks: the honesty vocabulary	16
8	Continuity	17
9	Identity	18
10	The TOHSENO mobile app and the Genesis invariant	21
11	The network	23
12	Public memory and the registry	26
13	Public and private data	28
14	Appcoins	29
15	The end-to-end system	31
16	A minimal protocol model	35
17	Security model	38
18	Sustainability	40
19	Protocol evolution and governance	40
20	The Genesis Shot	42

21	What TOHSENO is not, and the negative space	43
22	Constitutional invariants	44
23	Open questions	46
24	The wager	47
25	Closing	48

1 The name

TOHSENO is ONESHOT reversed.

The reversal matters.

“One shot” describes the act: a coherent intention is given a real attempt at becoming software.

The reversal describes the thesis: the conventional software process is turned around. Traditional software begins with infrastructure and eventually tries to recover the original human intention from requirements, tickets, meetings, and accumulated code. TOHSENO begins with the intention and grows only the infrastructure that reality proves necessary.

The name also encodes a discipline:

*Do not confuse caution with fragmentation.
When the ontology is clear, carry it through the whole system.*

TOHSENO is not a promise that every application will be perfect after one prompt. It is a promise that every coherent idea can receive an executable first form, then continue evolving without losing its identity.

And beneath the thesis sits the invitation the whole system is built to make easy, printed on the factory door and repeated at the end of every successful build:

Take another one.

2 The problem

2.1 Human intentions are abundant; software projects are scarce

People continuously imagine software that would improve their own lives: a tool for a family ritual; a game for a niche community; a strange personal instrument; a local business workflow; a training environment for a specific skill; an interface that reflects one person’s unusual way of seeing the world.

Most of these ideas never become software. Not because they lack value, but because the existing process assumes that software must justify the cost of becoming a project.

TOHSENO begins from the opposite assumption:

*Personal specificity is not a defect.
Weirdness is sufficient reason to create.*

2.2 Creation is becoming easier than continuity

Coding agents and increasingly capable development systems reduce the cost of producing a working application. That is necessary, but insufficient.

A generated application still faces harder questions. Who owns it? How does it remain understandable six months later? How does the builder experience it on a real device? How does feedback return to the place where the app evolves? How does one coherent app survive many subsequent instructions? How can another person test it before it reaches the App Store? How does the builder establish authorship? How do users carry identity across apps without being universally tracked? How can the app attach an economic object without binding itself to one launcher? What remains if the company operating the tool disappears?

A generator answers, “How do I produce code?”

TOHSENO asks, “How does an intention acquire a life?”

2.3 The precious-bet trap

There is a subtler failure mode than not building at all: building one thing and protecting it. When creating software is expensive, every idea must be treated as the one precious bet. It gets a pitch deck before it gets a body. It is defended before it is used. The builder learns to argue for the idea instead of learning from it.

TOHSENO takes the photographer’s position instead. Nothing has to be the one precious bet. The unit of progress is the roll, not the frame. A Shot is built to be used — and allowed to die. The contact sheet, not the pedestal.

2.4 Existing platforms capture the application

Many modern development platforms make creation easy by making the resulting software dependent on their runtime, account system, hosting, database, API keys, billing, or proprietary cloud. That convenience can turn into captivity.

TOHSENO takes the opposite position:

*The generated application belongs to the builder
before it belongs to the network.*

The network is additive. It provides continuity, not permission.

2.5 Identity is either fragmented or invasive

Most apps create their own accounts. The same human becomes a separate database row everywhere. The apparent alternative is a universal identifier exposed to every app, which enables effortless correlation and surveillance.

TOHSENO seeks a third model: one underlying identity per human; a different identity presented to each app; explicit, human-authorized proofs when identities should be linked. Continuity should not require automatic visibility.

2.6 Distribution and economics are disconnected from creation

An app can be technically complete and socially dead. App Store distribution is powerful but slow, selective, and downstream of substantial work. Open-source publication allows testing but lacks a coherent discovery and reputation layer. Tokens can create attention and economic motion, but token launch infrastructure is typically detached from the software artifact it claims to represent.

TOHSENO connects these without making any one of them mandatory.

3 The foundational primitive: the Shot

3.1 Definition

A **Shot** is one coherent software intention embodied as an independent application and preserved through its continuing lineage.

```
"Create a decision-training game for crypto traders."
-> one Shot
```

```
"Create an app that asks me to write before I scroll."
-> one Shot
```

```
"Create a tiny family app that remembers our bedtime stories."
-> one Shot
```

A Shot is not: a Git commit; a prompt; a version; a workspace containing several apps; a ticket; a release; a token; a child object beneath a larger “Project.”

Within the canonical TOHSENO ontology, the Shot is the application-level object.

3.2 The contact sheet

The word *Shot* is not a metaphor bolted on afterward. It is the ontology.

In photography, a shot is one exposure: a deliberate act of attention, committed to film, developed, and laid beside its siblings on a contact sheet. The photographer does not publish the contact sheet. They study it. Some frames get printed. Some teach the eye where to aim next. None of them are failures; all of them are evidence.

A builder’s collection of Shots is their contact sheet: a growing record of intentions given bodies.

The useful question about an idea is often not “is this worth months?” but “what does it feel like when it exists?” A Shot makes that question cheap to answer. Held next to its siblings, it does something a lone precious project never can:

*Volume reveals signal.
Taste emerges through making.*

Some shots live. Some teach you where to aim next. The roll continues.

3.3 One Shot, many evolutions

The first coherent intent creates the Shot. Later instructions may substantially transform it:

```
The Trenches
|-- first playable decision
|-- daily game
|-- post-game analysis
|-- rank
|-- community submissions
`-- App Store release
```

These are Evolutions of one Shot. They are not six Shots. The original circle remains around the same living object.

3.4 Stable identity

A Shot receives an immutable `shotId` when it is created.

The `shotId` must not be derived from mutable properties such as the app name, the current source tree, the latest intent, the bundle identifier, or the current owner. Names can change. Code can change. Ownership can change. The Shot remains the same.

A Shot ID should therefore be generated once using sufficient entropy or another collision-resistant method and retained in the Shot’s local manifest forever.

3.5 Independence

Every Shot must be independently operable.

A generated application must not secretly require: the TOHSENO CLI at runtime; a TOHSENO account; an official TOHSENO API; the TOHSENO mobile app; \$TOHSENO; a registry contract; or a proprietary build service.

A builder should be able to remove the global TOHSENO installation and continue working on the Shot as an ordinary source repository.

This is not an implementation preference. It is constitutional.

3.6 Ejectable from birth

Independence has a stronger form, and the factory holds itself to it: **ejection**. Ejection means the owner can build, operate, modify, migrate, or replace a Shot without TOHSENO's permission or a hidden factory dependency — and it is true at creation time, not a later export feature.

Open source alone is insufficient. A repository can be public and still captive. Ejection is made real by concrete properties every Shot carries from its first commit: reproducible source; its own ordinary Git history; pinned, locally runnable checks; explicit data and authority declarations; owner-held identifiers; and no symlinks into a checkout, cache, installation, or network package.

A Shot is *not* ejectable if its core action requires a TOHSENO account, endpoint, secret, CLI, mutable global validator, factory cache, or undisclosed service; if build assets are linked out of the repository; if only TOHSENO can read canonical data; or if leaving requires publishing private input.

The factory's promise is symmetric: every Shot can leave, and the builder can always take another one.

4 Intent and Evolution

4.1 Intent

An **Intent** is a human-readable statement of desired change.

The first intent answers: *What should exist?*

A later intent answers: *Given what reality taught me, what should this Shot become next?*

An intent may be created from a direct instruction; feedback from a user; observation of the app in use; a failed verification; a new design decision; a change in the builder's taste; a market signal; a conversation; or a constraint discovered on a real device.

The intent is private by default. It belongs to the builder's workshop, not to the network, not to a provider's logs, and not to the generated repository's public history. What crosses any boundary is a deliberate, sanitized projection — never the raw thought.

4.2 Evolution

An **Evolution** is a meaningful application of intent to an existing Shot.

Not every file change is an Evolution. A formatter pass, dependency update, or incidental commit may be part of an Evolution without constituting one by itself. An Evolution marks a meaningful step in the app's life: a change the builder would recognize as an answer to a real intention. An Evolution is counted only when two independent judgments agree: the coding agent completed its work, and deterministic verification passed. Conceptually, an Evolution may contain:

| shotId

```

evolution number
intent or private intent commitment
previous evolution hash
source tree hash
manifest hash
composition lock hash
artifact hash, when applicable
verification result
created time
builder signature, when made public

```

The local repository may use Git as its detailed technical history. TOHSENO's Evolution history is the semantic history layered above it.

4.3 The making loop

TOHSENO is not complete when code appears. The full loop is:

```

intention
  |
  v
Shot
  |
  v
use on a real device
  |
  v
felt experience
  |
  v
feedback
  |
  v
sharpened next intent
  |
  v
Evolution
  |
  v
use again

```

The purpose of continuity is to keep this loop intact. The app becomes a conversation between intention and reality.

A shot becomes useful through contact with the real app, not through terminal output alone. Ideas become true in use. And the loop has one more arm, easily forgotten and deliberately honored: *keep, evolve, or kill it — then take another shot*. Ending a Shot well is a legitimate exit from the loop. Built to be used. Allowed to die.

5 Lifecycle

5.1 Canonical states

Every Shot has one canonical distribution state:

| EVOLVING -> PUBLISHED -> APP_STORE

The state is monotonic in public history. A later record may deprecate availability, transfer control, or mark a release obsolete, but it must not erase the fact that a Shot was previously published or released.

5.2 EVOLVING

A Shot is **EVOLVING** while it is primarily local to its builder.

It may be fully functional. It may run on a simulator or physical device. It may have many Evolutions. It may even be shared privately. EVOLVING does not mean unfinished in a pejorative sense. It means the Shot remains inside the builder's workshop.

Properties: private by default; no network identity required; no public source required; no blockchain required; continued evolution expected.

5.3 PUBLISHED

A Shot is **PUBLISHED** when a verifiable source representation is made available through the TOHSENO ecosystem so that other people can obtain and run it.

Published source should include enough information to answer: Which Shot is this? Which Evolution is this? Where is the source? Which source tree was published? How can it be built? Under what license is it available? Which builder signed or claimed the publication?

Publication does not mean App Store release. It creates a space between private development and centralized distribution where other people can inspect the source, build the app, run it on their own device, test it, provide feedback, fork it under its license, and verify its lineage.

A node may index and mirror a published Shot, but the node is not the publisher. The builder publishes. The node witnesses, serves, and relays.

5.4 APP_STORE

A Shot enters **APP_STORE** when it is released through Apple's App Store. In human language, this is the Shot being **released**.

An App Store record may include:

| Apple app identifier

```

bundle identifier
version
source evolution
release artifact hash, when available
store URL
verification method
release time

```

APP_STORE is not the end state of creativity. A Shot may continue to receive Evolutions forever while retaining APP_STORE as its highest distribution milestone.

The road from a built Shot to a live listing is walked by a human: an Apple developer account, signing, a listing, review, and one step that is genuinely irreversible — *submit for review; this one you cannot take back*. TOHSENO may describe that ladder precisely, detect progress honestly, and wait patiently. It must never pretend the ladder is shorter than it is, and it must make irreversible rungs quieter, slower, and clearer than ordinary steps.

5.5 Lifecycle and recording are different

A Shot's lifecycle state and its network recording status are orthogonal. An EVOLVING Shot may record a private hash commitment proving that it existed at a certain time. A PUBLISHED Shot may remain entirely off-chain. An APP_STORE Shot may use the network only for feedback. A Shot may have a public builder record without an appcoin, or link an appcoin after years of existence.

This distinction prevents the registry from becoming a gatekeeper.

6 The local factory

6.1 The factory is the base layer

TOHSENO must begin as a complete local tool. The expected experience is conceptually simple:

```

install tohseno
tohseno          # take the first shot
tohseno create
tohseno evolve
tohseno verify
tohseno run
tohseno publish

```

The exact command surface may evolve, but the principle does not:

A person can create and carry a Shot through its full software lifecycle without joining the TOHSENO network.

Speed is the product. The mission is intention to phone, measured in minutes. Anything that adds a question, a config step, or a ceremony must pay for itself in reliability.

6.2 One intention, explicit interpretation

The factory divides responsibility along a single, strict boundary:

*Artificial intelligence interprets meaning.
Deterministic code owns composition, locks, storage invariants,
verification, and authority.*

The builder gives one private intention. A coding agent — owner-selected and replaceable — reads it once, in a sandbox with no approvals and no network write authority, and returns a small, sanitized, validated plan: a name, a starting shape, a set of installed capabilities, a data posture, an identity posture, a first definition of done. The plan is shown to the human for approval before anything is built. If the agent is offline, slow, or wrong, the factory falls back to a blank starting shape — never to another provider chosen on the builder's behalf.

From the approved plan onward, determinism takes over. Composition is a locked overlay of a neutral kernel, a starting template, and dependency-ordered capabilities, fingerprinted file by file. The repository is materialized atomically: manifest, composition lock, pinned verifier, provenance, Git baseline — all created and verified before the directory becomes visible. The agent then works *inside* that already-true repository, with a sanitized environment and bounded authority, and its result is verified again after it exits. A failing result is isolated, not presented as ready.

Capabilities are installed, not implied. Prose can guide an agent, but prose cannot silently claim an uninstalled capability. If a feature cannot be expressed as a valid manifest field, it is unsupported — the factory says so instead of improvising.

6.3 Agent agnosticism

TOHSENO may coordinate coding agents, models, compilers, templates, and verification tools, but it must not define the identity of a Shot by which model produced it.

A Shot can outlive the model used to create it, the agent harness, the template version, the TOHSENO release, and the company operating any of those systems. The protocol should record relevant composition fingerprints without making a temporary intelligence provider the center of the ontology.

6.4 Truthful composition

A Shot should carry enough local metadata to reconstruct how its current form came to exist: app manifest; TOHSENO protocol version; template identifiers; skill descriptors; lockfiles; immutable file hashes; verifier results; source tree hashes; optional private provenance.

Private provenance must remain private by default. The public network record is a deliberate projection of local truth, not a copy of the entire workshop.

6.5 Verification before ceremony

TOHSENO should prefer executable checks over confidence language.

A Shot that claims to build should build. A Shot that claims to be published should expose resolvable source. A Shot that claims an App Store release should point to a verifiable listing. A Shot that claims an appcoin should identify the chain, address, and deployment transaction.

The system should present verification levels honestly:

```
creator claimed
signature verified
source fingerprint verified
source available
release discovered
release control verified
token contract verified
deployment transaction verified
```

Not all claims have equal strength. TOHSENO must never blur that distinction.

The same honesty governs the moment of handoff. When a Shot is born, the factory — not the agent — delivers the report: what was verified, what was built, what launched, where the screenshot landed. The agent builds, but TOHSENO is the only party allowed to say it worked, because its statement is backed by executable evidence rather than enthusiasm. The report ends with exactly one next action, and the reason for it:

```
NEXT
Use the app in Simulator and decide what its
next Evolution needs.

    tohseno my-app

WHY
A shot becomes useful through contact with the real app,
not through terminal output alone.
```


6.6 Open underneath, magic at the surface

The factory's interior is radically inspectable: open source, pinned versions, content-addressed releases, checksummed installs that re-verify themselves on every invocation, and a repository gate that mechanically enforces the system's own promises. Its surface is one line, one intention, one working app.

These are not in tension. The magic is trustworthy precisely because the underneath is boring, deterministic, and public. Transparent underneath. Frictionless at the surface.

7 How this document speaks: the honest vocabulary

A whitepaper written before a stable release faces a temptation: to describe the dreamed system and the running system in the same tense. TOHSENO refuses that tense. Throughout the project — and throughout this document — claims are classified with a four-word vocabulary:

IMPLEMENTED

Exercised by a live code path and testable now.

PREPARED

Configuration and instructions exist, but no external action has occurred.

PROPOSED

Architecture or product behavior that is not implemented.

OPEN

Requires a product, security, or ownership decision.

Applied to this document, at the moment of writing:

The Shot ontology, deterministic composition and locks, atomic materialization, agent-boundary planning with blank fallback, pinned verification, Evolution counting, Simulator run and capture, the Studio, the checksummed installer, and protocol version 1's signed record schemas with a reference node are **IMPLEMENTED**.

The human ladder from built Shot to App Store listing, and the reference-node runbook, are **PREPARED**.

Publication records flowing from real builders, deployed nodes, persistent builder identity, the mobile app, registry anchoring, and appcoin links in the wild are **PROPOSED**.

The production trust root, cross-node fork resolution, anchoring economics, the commercial boundary, and several questions collected in §23 are **OPEN**.

Stale confident instructions are worse than an honest unknown. A future reader comparing this document against a future system should be able to tell exactly which promises were running, which were staged, and which were still dreams — and hold the project to the difference.

8 Continuity

8.1 Definition

Continuity is the ability of a Shot, a builder, and a user relationship to survive changes in device, version, agent, node, host, and distribution channel without surrendering sovereignty.

Continuity has three dimensions:

1. **Artifact continuity.** The Shot remains the same living object across Evolutions and releases.
2. **Human continuity.** A person can carry identity, consent, and selected relationships across apps and devices.
3. **Network continuity.** Public records and published artifacts can remain available beyond one official server.

8.2 A continuity app

A **continuity app** is an independent application that may optionally participate in TOHSENO's continuity protocols. Participation may allow an app to request: an app-specific user identity; a signed login challenge; explicit cross-app linkage; feedback delivery; entitlement proofs; recovery; a public Shot record; appcoin information; connection to a builder's next-intent loop.

An app remains functional if the user declines all of these.

The continuity layer is a capability, not a tax.

8.3 The continuity paradox

TOHSENO seeks two properties that are often treated as opposites:

	the same human across many apps
	and
	no automatic universal identifier across those apps

The system resolves this through derived, app-specific identities and explicit proofs of sameness. Continuity without surveillance is not an optional privacy feature. It is the identity model.

9 Identity

9.1 There is no global seed phrase

There is no global TOHSENO seed phrase shared by builders or users. Every human controls an independent root.

```
JP root      != Maria root
Alice root   != Bob root
```

No company-wide secret, protocol-wide mnemonic, or shared developer key may stand in for personal identity.

9.2 The Continuity Root

A person may create a **Continuity Root**: a private cryptographic root from which scoped identities can be derived. The root may be held by the future TOHSENO mobile app; a local desktop signer; a hardware wallet; a compatible third-party signer; or another open-source implementation.

The TOHSENO mobile app is a preferred premium custodian, not a protocol monopoly. The root secret must never be transmitted to a node or generated app.

9.3 Builder identity

A builder may derive or designate a stable public identity for authorship. This identity can sign claims such as:

```
I created this Shot.
I authorize this Evolution as canonical.
I publish this source fingerprint.
I acknowledge this App Store release.
I link this appcoin to this Shot.
I transfer current control of this Shot.
```

Builder identity is designed to accumulate public reputation. A builder record may eventually show Shots created, Shots published, App Store releases, verified source histories, community feedback, appcoin relationships, node attestations. The purpose is not gamification for its own sake. It is a credible body of work — a public contact sheet.

9.4 User identity

When a person uses a continuity app, the signer derives an app-specific identity. Conceptually:

```
APP_KEY = derive(
  CONTINUITY_ROOT,
  protocol domain,
  app identifier,
  environment
)
```

The same person receives different identities in different apps:

```
Alice root
|-- Alice-in-Anky
|-- Alice-in-The-Trenches
`-- Alice-in-Garden-Memory
```

Anky cannot automatically infer that Alice-in-Anky and Alice-in-The-Trenches are the same person. This prevents the continuity root from becoming a universal tracking number.

9.5 Explicit cross-app proofs

A person may choose to prove that two app-specific identities belong to the same root. The signer presents a human-readable request:

```
THE TRENCHES wants to confirm
that you also use ANKY.

This links your identities between these two apps.

No other app identities will be revealed.
```

Only after explicit approval does the signer produce a scoped continuity proof. The proof should be specific to the requesting apps; specific to the requested purpose; time-bounded where appropriate; revocable where the claim is not permanently public; and incapable of revealing the root secret.

9.6 Builder and user roles

The same human may be both a builder and a user. Those roles remain distinct:

```
JP root
|-- public builder identity -> created Anky
`-- private app identity    -> JP-in-Anky
```

An app does not automatically receive the builder's public identity merely because the builder uses it.

9.7 Collaboration

Authorship, contribution, and control are different concepts. A Shot may eventually support one genesis builder; several co-creators; credited contributors; a current controller; an organization or multisignature controller; a publisher-of-record for an App Store account.

Original authorship is historical and must not be silently transferred. Current control may change. A future owner can control releases without becoming the person who originally created the Shot.

9.8 Recovery and rotation

Continuity requires recovery, but recovery must not create an invisible custodial backdoor. Compatible recovery methods may include encrypted recovery packages; backups secured by the platform; user-chosen recovery contacts; social recovery; key rotation signed by the current key; and recovery policies held in contract accounts.

A rotated key may assume current control while old authorship signatures remain valid historical facts.

10 The TOHSENO mobile app and the Genesis invariant

10.1 What it is

The future TOHSENO mobile app is an optional premium continuity client. It may become: a continuity identity vault; a hardware-like signer; a mobile approval surface; a portfolio of the builder's Shots; a place to experience connected apps as a user; a structured feedback inbox; a next-intent sharpening instrument; a bridge between real device use and the local Studio; a hosted-network convenience layer; a subscription product.

It is not the place where the source code fundamentally belongs. The development machine remains the factory.

10.2 What it is not

The mobile app is not: required to install TOHSENO; required to create, evolve, or publish a Shot; required to ship to the App Store; required for a generated app to function; the only valid signer; the root owner of generated apps; a mandatory wallet; or the protocol itself.

A builder must be able to use the complete open-source factory without ever seeing the TOHSENO mobile app.

10.3 Mobile authorization

When paired with a local Studio, the phone may receive a readable request:

RECORD THIS SHOT

THE TRENCHES
Evolution 7

This will permanently publish:

- + Shot identifier
- + Builder identity
- + Source fingerprint
- + Lifecycle state

This will not publish:

- x Original private intent
- x Source files not already published
- x Agent conversation
- x Local secrets

The builder may authorize with a deliberate gesture such as swiping right. The gesture is not decoration. It represents:

*I have seen the claim.
I understand its permanence.
Make it mine in public memory.*

10.4 Feedback and the next intent

The mobile app's most important product role may be the feedback loop. A builder can use a Shot on the actual device where it matters, then capture what felt wrong; what surprised them; what the app appears to want to become; what another tester said; what should change next.

The mobile app can help compress this experience into a precise next intent and send it to the local Studio. This is premium continuity:

```
the app remains close enough to life
that its next evolution is informed by use,
not by memory of what the builder thought before using it.
```

10.5 Subscription boundary

TOHSENO may charge monthly or yearly for the mobile experience and hosted convenience: synchronized builder identity; secure mobile signing; cross-device recovery; hosted public builder profile; feedback routing; intent sharpening; community discovery; node relay and gas sponsorship; premium network indexing; continuity across connected apps.

It must not purchase the right to own, build, or continue operating one's own source code.

TOHSENO charges for continuity, not freedom.

10.6 The genesis invariant

The TOHSENO mobile app must not be implemented by hand before the first stable release of the protocol. The first stable TOHSENO system must create it through the same public factory offered to every builder.

Genesis is generated. The mobile application's absence from the factory's own repository is deliberate, and that absence is a tested property. Building the app by hand, checking its source into the factory first, or adding a privileged template that bypasses the ordinary path would invalidate the proof that the factory can create its own product. The mobile Shot's repository is an *output* of the stable release, not an input to it.

The intended sequence is:

```
stabilize the TOHSENO factory and protocol
|
```

```

      v
run the stable `tohseno create`
      |
      v
create the TOHSENO mobile app as the Genesis Shot
      |
      v
evolve it through the ordinary lifecycle
      |
      v
publish its source
      |
      v
record its lineage
      |
      v
link its canonical appcoin
      |
      v
release it on the App Store

```

No hidden template may special-case the Genesis Shot.

The loom must be complete before it weaves its first flag.

11 The network

11.1 The network is optional

The network surrounds the local factory but does not replace it.

LOCAL FACTORY	OPTIONAL NETWORK
-----	-----
create	record
evolve	mirror
verify	discover
run	route feedback
own	relay
publish source	link appcoin
ship	preserve public memory

If every TOHSENO node disappears, a builder's local Shots must remain usable. Nodes index; builders own.

11.2 The node

A **Node** is a compatible service that performs one or more network functions. A node may: receive signed public records; validate schemas; verify signatures and hashes; store or pin public manifests; mirror published source; index registry events; serve builder and Shot profiles; route encrypted feedback; relay transactions; sponsor gas; verify appcoin deployments; verify public release claims; synchronize public records with peers.

A node does not become the author, owner, or publisher merely by serving a Shot. A node is an index and a transport, not an ownership authority or a consensus system.

- The builder authors.
- The builder publishes.
- The node witnesses and serves.
- The registry remembers.

11.3 The official node

The first official TOHSENO service may run a minimal server. It can provide the default experience, but all clients should depend on a documented node interface rather than hardcoding one domain — a rule the factory enforces mechanically against its own code. Conceptually:

```
interface TohsenoNode {
  submitRecord(record: SignedRecord): Promise<RecordReceipt>;
  getShot(shotId: ShotId): Promise<NetworkShot | null>;
  getBuilder(id: PublicIdentity): Promise<BuilderRecord | null>;
  linkAppcoin(link: SignedAppcoinLink): Promise<LinkReceipt>;
  routeFeedback(payload: EncryptedFeedback): Promise<DeliveryReceipt>;
}
```

A user should eventually be able to choose:

```
tohseno network use https://node.tohseno.com
tohseno network use https://community.example
tohseno network use http://localhost:7036
```

The abstraction is small by design. It leaves the door open to decentralization without pretending decentralization already exists.

11.4 Objective verification, not aesthetic judgment

Nodes may verify objective properties: the signature matches the claimed builder; the manifest hashes to the claimed value; the source tree matches the public source; the parent record exists; the token exists at the claimed address; the deployment transaction created the claimed token; the App Store identifier resolves; the record follows the protocol schema.

Nodes should not initially decide whether an app is good; whether an idea is original enough; whether the creator “deserves” authorship; whether a token is economically wise; whether an app is culturally important.

TOHSENO records claims and evidence. It does not claim omniscience.

11.5 Existing consensus, not a new blockchain

TOHSENO does not need to invent its own blockchain to decentralize nodes. An existing chain can provide ordering, timestamps, transaction finality, and globally readable events. Nodes provide the surrounding work: storage, indexing, verification, availability, relaying, discovery.

This keeps the protocol narrow.

11.6 Progressive decentralization

The likely path is:

Stage 0 — Local-only.

The factory works with no node.

Stage 1 — Official node.

Anky, Inc. or another initial steward operates the first node.

Stage 2 — Community mirrors.

Anyone can run a node that syncs public records, mirrors manifests, and serves discovery.

Stage 3 — Active relayers.

Multiple authorized or bonded nodes can validate and relay new records.

Stage 4 — Economic coordination.

Nodes may stake and earn \$TOHSENO for measurable, verifiable work.

This sequence matters. A token must not subsidize empty infrastructure in the hope that meaningful applications eventually appear.

The organism must breathe before its metabolism is financialized.

12 Public memory and the registry

12.1 The purpose of the registry

The registry answers small, durable questions:

```
Did this builder claim this Shot?
Did this Evolution belong to this Shot?
Did the builder publish this source fingerprint?
Did this Shot reach a lifecycle milestone?
Which appcoin did the builder declare canonical?
Who currently controls official updates?
What was the public history of those claims?
```

It does not store the entire application.

12.2 Append-only facts

The registry is append-only. A record may supersede an earlier record, but must not erase it. This permits key rotation, appcoin migration, control transfer, deprecation, corrected metadata, and new lifecycle milestones. History remains visible.

Records are portable builder attestations, not network truth. An honest registry knows the difference and says so.

12.3 Record types

A minimal protocol may define records such as:

```
SHOT_CLAIM
EVOLUTION_CLAIM
LIFECYCLE_TRANSITION
APPCOIN_LINK
CONTROL_TRANSFER
DEPRECATION
```

The exact encoding may evolve through protocol versions, but the semantic boundaries should remain clear. Unknown fields fail validation. There are deliberately no fields for raw prompts, private provenance digests, local databases, app-user content, conversations, credentials, secrets, unpublished source bytes, or model reasoning. A safe-looking summary can still disclose an idea, so signing and submitting a record must remain a deliberate public action.

12.4 A generic record

Conceptually:

```
{
  "protocol": "tohseno",
  "protocolVersion": 1,
  "recordId": "0x...",
  "recordType": "LIFECYCLE_TRANSITION",
  "shotId": "0x...",
  "previousRecordId": "0x...",
  "actor": "0xBUILDER...",
  "manifestHash": "0x...",
  "manifestURI": "ar://...",
  "createdAt": "2026-07-26T16:00:00Z"
}
```

The recordId should commit to the exact canonical representation. The human-readable manifest may live in a TOHSENO node, object storage, IPFS, Arweave, or another content-addressed system. The onchain record stores a hash and a pointer.

12.5 Creator authorization

A public claim should be authorized by the builder or current controller. A typed-data signature can bind: protocol domain; registry; chain; record type; Shot ID; manifest hash; nonce; expiration. The signature should be human-readable before approval. A node may submit and pay gas without becoming the creator.

12.6 Gasless experience

The preferred experience is:

```
| review -> sign -> recorded
```

Not:

```
| buy ETH -> bridge -> select network -> estimate gas -> retry
```

A node may sponsor gas for eligible records. A builder may also self-relay or use another node. Gas sponsorship is a service, not a protocol dependency.

12.7 Why the registry is not an NFT

The permanent authorship record should not begin as a transferable NFT. Authorship, control, ownership, and collectibility are different concepts. An ordinary transferable NFT can imply: *whoever holds this token is the creator or owner of the represented app*. That may be false.

The registry should preserve: *this identity created or authorized this public record at this time*. Future transferable project-control objects or collectibles may exist on top of that history, but they must not rewrite authorship.

12.8 Public record as career memory

The registry is useful because it can become the memory of a builder's life in software. A credible profile may eventually show:

- the Shots a person created
- how long they carried them
- which ones reached publication
- which ones reached the App Store
- which appcoins were linked
- which source histories were verified
- which communities formed around them

*Provenance is not paperwork.
It is a body of work that can survive a platform.*

13 Public and private data

13.1 Local truth, public projection

The local Shot contains richer information than the public network record. The public record must be derived intentionally. A publication preview should state exactly what crosses the boundary.

13.2 Public by explicit choice

Possible public fields: Shot ID; app name; icon; builder public identity; lifecycle state; Evolution number; source locator; source tree hash; manifest hash; build instructions; license; App Store identifiers; appcoin chain and address; public screenshots; public description.

13.3 Private by default

Private fields include: unpublished source; original private intentions; agent conversations; local databases; secrets; signing keys; user data; private feedback; internal analytics; device identifiers; recovery material.

Private-by-default and account-free are defaults, not refusals. A manifest may truthfully declare other data, identity, integration, entitlement, or irreversible mechanics. Those are product decisions the builder makes in the open — streaks, paywalls, and scores are tools, not sins — and the manifest exists so that such declarations are explicit rather than implied.

13.4 Hashes are not automatically private

A hash of low-entropy text can leak information through guessing. Private intentions should use appropriate commitments, including salts or keyed constructions, when later proof is desired. TOHSENO must not call data “private” merely because it has been hashed.

13.5 Irreversibility requires legibility

Before any permanent record, the user must be able to understand: what will become public; what will remain private; which identity signs; whether a node will sponsor gas; whether the record can be superseded; whether the action can ever be forgotten.

The protocol should make irreversible actions quieter, slower, and clearer than ordinary interface taps.

14 Appcoins

14.1 Definition

An **Appcoin** is an optional economic artifact canonically linked to a Shot. It may represent a community’s shared attention; a way to fund continued development; an instrument for distribution; access, membership, or coordination; a market around an app’s cultural motion; an experiment whose meaning evolves with the Shot.

An appcoin is not the Shot. The Shot can exist without a coin. The coin should point back to a Shot.

Recording a link does not deploy, mint, trade, transfer, price, or endorse an asset. TOHSENO never promises wealth, and never makes financial mechanics, urgency, or speculation the reason to build. The prototype is the payoff; the coin, when it exists, is downstream of a living thing.

14.2 Deployment agnosticism

TOHSENO must not require appcoins to be deployed by the registry contract. A coin may be deployed through any launcher, a custom smart contract, a future chain, or a mechanism not yet invented. The TOHSENO record links the canonical relationship:

- Shot ID
- coin chain ID
- token address
- deployment transaction hash
- deployment provider, if known
- linking builder signature
- link time

| economic metadata

The protocol cares about the relationship, not the launcher.

14.3 Canonical appcoin

A Shot may have zero or one active canonical appcoin at a time. If a coin migrates, the new link explicitly supersedes the prior one; the old relationship remains historical. Because Evolutions belong to the same Shot, an appcoin does not need to be recreated for every version.

14.4 Verification in both directions

A strong appcoin relationship can point both ways:

| TOHSENO record -> token address
| token deployment metadata -> TOHSENO Shot ID

A one-way builder claim is still useful, but a bidirectional link is stronger.

14.5 Fee capture

A deployment-agnostic registry cannot magically collect fees from arbitrary token contracts. Any fee routing must be explicit. Possible future models include a launch adapter that routes a declared portion of trading fees; a token contract that names builder, launcher, and network beneficiaries; a voluntary protocol fee; a node relay fee; a subscription-funded public service; app-specific economic rules.

TOHSENO must never claim revenue rights that are not encoded and consented to.

14.6 \$TOHSENO

The intended genesis is for \$TOHSENO to become the first canonical appcoin recorded by the stable protocol and linked to the TOHSENO Genesis Shot. Its deployment mechanism remains replaceable. Its potential roles are distinct:

1. **Appcoin of the TOHSENO mobile app.** It is economically connected to the first canonical app created by the system.
2. **Coordination asset for the network.** It may eventually be staked or earned by nodes performing verifiable work.
3. **Shared economic surface of the ecosystem.** Future appcoin adapters may choose to route explicit value toward the network or \$TOHSENO.

None of these roles may make \$TOHSENO mandatory for local creation, ordinary app use, source ownership, or App Store release.

14.7 Node rewards

If node rewards are introduced, they should compensate measurable work: storing and serving manifests; mirroring published source; validating records; relaying transactions; indexing events; maintaining availability; providing cryptographic proofs of service.

Rewards should not be issued merely because a process claims to be online. Otherwise, the network invites empty node farming rather than useful infrastructure. Tokenomics should describe real metabolism, not substitute for it.

15 The end-to-end system

15.1 Local creation

IMPLEMENTED

```

Builder writes first intent
  |
  v
agent interprets it into a sanitized, approvable plan
  |
  v
TOHSENO composes deterministically and creates a Shot
  |
  v
Shot receives immutable Shot ID
  |
  v
independent source repository, atomic from birth
  |
  v
local run, verification, and Simulator capture

```

No account, node, mobile app, token, or chain is involved.

15.2 Evolution

IMPLEMENTED

```

Builder uses the app
  |
  v
captures feedback
  |
  v
sharpens next intent
  |
  v
TOHSENO applies an Evolution

```



```

      |
      v
verification runs
      |
      v
same Shot, new form

```

15.3 Publication

PROPOSED

```

Builder chooses PUBLISHED
      |
      v
TOHSENO prepares public projection
      |
      v
source tree and manifest are fingerprinted
      |
      v
builder signs locally or with mobile signer
      |
      v
source is made available
      |
      v
any compatible node can index or mirror it

```

The builder may publish source without an onchain record.

15.4 Permanent recording

PROPOSED

```

factory prepares readable claim
      |
      v
signer shows exact public fields
      |
      v
human authorizes
      |
      v
node validates signature and schema
      |
      v
node stores manifest
      |
      v
node relays hash to registry
      |

```

```

      v
public receipt returns

```

15.5 App Store release

PREPARED

```

Shot reaches Apple release
|
v
release metadata is captured
|
v
builder signs APP_STORE transition
|
v
node verifies available evidence
|
v
registry records milestone

```

The Shot continues evolving afterward.

15.6 Appcoin link

PROPOSED

```

builder deploys coin anywhere
|
v
TOHSENO receives chain, address, deployment transaction
|
v
builder reviews canonical link
|
v
builder signs
|
v
node verifies token and transaction
|
v
registry appends APPCOIN_LINK

```

15.7 User continuity

PROPOSED

```

User opens a continuity app
|
v
app requests scoped identity

```

```
      |
      v
compatible signer derives app-specific key
      |
      v
user approves
      |
      v
app receives proof, not root secret
```

15.8 Cross-app continuity

PROPOSED

```
App A requests proof of relationship to App B
      |
      v
signer shows exact scope
      |
      v
user approves
      |
      v
signer creates pair-specific proof
      |
      v
only the requested identities become linkable
```

16 A minimal protocol model

The following model is illustrative, not a frozen wire format.

16.1 Local Shot manifest

```
{
  "schema": "tohseno.shot/v1",
  "shotId": "shot_01...",
  "name": "The Trenches",
  "createdAt": "2026-07-26T16:00:00Z",
  "lifecycle": "EVOLVING",
  "currentEvolution": 7,
  "platform": "ios",
  "bundleId": "app.thetrenches.mobile",
  "protocolVersion": "1.0.0",
  "composition": {
    "template": "ios-native",
    "skillsLockHash": "0x...",
    "sourceTreeHash": "0x..."
  },
  "network": {
    "recorded": false,
    "canonicalNode": null
  }
}
```

16.2 Evolution manifest

```
{
  "schema": "tohseno.evolution/v1",
  "shotId": "shot_01...",
  "evolution": 7,
  "previousEvolutionHash": "0x...",
  "intentCommitment": "0x...",
  "sourceTreeHash": "0x...",
  "manifestHash": "0x...",
  "artifactHash": null,
  "verification": {
    "status": "passed",
    "reportHash": "0x..."
  },
  "createdAt": "2026-07-26T16:30:00Z"
}
```

16.3 Public Shot record

```
{
  "schema": "tohseno.public-record/v1",
  "recordType": "LIFECYCLE_TRANSITION",
  "shotId": "shot_01...",
  "evolution": 7,
  "builder": "0x...",
  "lifecycle": "PUBLISHED",
  "source": {
    "uri": "https://...",
    "treeHash": "0x...",
    "license": "..."
  },
  "manifestHash": "0x...",
  "previousRecordId": "0x...",
  "createdAt": "2026-07-26T16:45:00Z"
}
```

16.4 Appcoin link

```
{
  "schema": "tohseno.appcoin-link/v1",
  "shotId": "shot_01...",
  "builder": "0x...",
  "chainId": 8453,
  "tokenAddress": "0x...",
  "deploymentTxHash": "0x...",
  "deploymentProvider": "bankr",
  "supersedes": null,
  "createdAt": "2026-07-26T17:00:00Z"
}
```

16.5 Signing request

```
{
  "schema": "tohseno.signing-request/v1",
  "action": "LINK_APPCOIN",
  "humanSummary": {
    "title": "Link official appcoin",
    "shotName": "The Trenches",
    "publicFields": [
      "Shot identifier",
      "Builder identity",
      "Chain ID",
      "Token address",
      "Deployment transaction"
    ]
  }
}
```

```

    ]
  },
  "payloadHash": "0x...",
  "expiresAt": "2026-07-26T17:10:00Z"
}

```

16.6 Node receipt

```

{
  "schema": "tohseno.node-receipt/v1",
  "node": "node_public_key",
  "recordId": "0x...",
  "manifestHash": "0x...",
  "storage": {
    "uri": "ar://...",
    "status": "stored"
  },
  "registry": {
    "chainId": 8453,
    "transactionHash": "0x...",
    "confirmations": 12
  },
  "receivedAt": "2026-07-26T17:01:00Z"
}

```

17 Security model

17.1 Nodes are not trusted with identity secrets

A node receives public keys, signed claims, public manifests, and encrypted payloads. It must never receive the Continuity Root; app-specific private keys; recovery phrases; unencrypted private feedback unless explicitly addressed to the node as a service; or unpublished source by default.

17.2 Generated apps are not trusted with the root

A generated app receives an app-specific proof or asks the signer to sign a challenge. It should not receive the Continuity Root. A compromised experimental app must not compromise every other continuity app.

17.3 Agents are not trusted with the factory

The factory extends the same discipline to the coding agents it coordinates. An agent plans in a read-only sandbox and builds inside a repository whose protected state is snapshotted before and verified after its run. Its environment is sanitized: credentials the factory happens to inherit are never forwarded merely because the agent would find them convenient. An agent's claim of success is not accepted; it is checked. A result that fails verification is isolated, not shipped.

17.4 Domain separation

Derived keys and signatures must use explicit domains. A login signature must not be replayable as an appcoin authorization. A signature for one registry or chain must not silently authorize another. A test environment must not share production authority.

17.5 Replay protection

Permanent and temporary signing requests should include appropriate nonces, deadlines, record IDs, previous-record references, and chain and contract domains.

17.6 Canonical encoding

Hashing requires deterministic encoding. The protocol must define field order; Unicode normalization; number representation; timestamp representation; omission versus null; URI normalization; canonical JSON or another deterministic format. Without canonicalization, two honest implementations can disagree about the same record.

17.7 Local-first does not mean careless

Local keys require secure system storage; explicit export; backups; rotation; clear environment separation; no plaintext secrets in repositories. The mobile signer may improve key safety, but must not be the only safe path.

17.8 Supply chain honesty

The path from “curl” to “factory” is part of the security model. An installer should pin exact versions and checksums; refuse insecure transport; refuse to mutate installations it cannot authenticate; and leave behind a tool that re-verifies its own integrity every time it runs. Magic at the surface is only tolerable when the surface can prove what is underneath.

17.9 Open-source verification

The security of TOHSENO should not depend on users trusting a closed official client. Protocol schemas, signature logic, registry contracts, and reference node behavior should be inspectable.

17.10 Registry humility

An onchain timestamp proves that a signed claim existed by a certain block. It does not automatically prove the builder was the first human ever to imagine the idea; the source was free of copied work; the App Store account was legally owned; the token was safe; or the app was valuable.

TOHSENO must state exactly what each proof establishes and no more.

18 Sustainability

18.1 What remains free

The following must remain available without a subscription: installing the open-source factory; creating Shots; evolving Shots; running and verifying source; owning generated repositories; publishing source through compatible mechanisms; using alternative signers; using alternative nodes; self-hosting; shipping independently.

18.2 What can be paid

TOHSENO may charge for services that create real ongoing cost or convenience: mobile continuity; hosted identities and recovery assistance; synchronized feedback; sharpened next-intent workflows; official-node storage; relaying and sponsored gas; public profiles; discovery; community testing; managed publication; managed App Store operations; premium agent orchestration; organization features.

The business should grow around the open protocol rather than weakening it.

18.3 Builder-aligned economics

If TOHSENO participates economically in appcoins, the relationship must be explicit. The builder should be able to see who receives fees; why; at what rate; through which contract or provider; and whether the relationship can change. Hidden extraction would contradict the protocol's purpose.

18.4 The network as a service before a market

Before nodes receive token rewards, the official node should prove that builders want public records; users want published apps; feedback actually returns; storage and relay work are measurable; and independent nodes can reproduce the behavior.

19 Protocol evolution and governance

19.1 Versioned specifications

TOHSENO protocol artifacts must be versioned:

- | shot schema
- | evolution schema
- | public record schema

- signing protocol
- node API
- identity derivation
- registry contract
- continuity SDK

Old records remain interpretable forever.

19.2 Backward compatibility

A new TOHSENO release should prefer additive fields; explicit migrations; versioned endpoints; stable record semantics; preserved local operability. A future agent must not silently reinterpret an old PUBLISHED record to mean something new.

19.3 Constitutional changes

Some implementation details may change freely. The core invariants require exceptional scrutiny. A protocol version that makes the mobile app mandatory, turns the official node into a gatekeeper, exposes a universal user identifier, or makes generated apps dependent on TOHSENO at runtime is not a routine upgrade. It is a different system.

19.4 Stewardship

The initial repository and official infrastructure may have a central steward. That stewardship is practical, not metaphysical. The long-term credibility of TOHSENO comes from open specifications; independent implementations; portable signatures; content-addressed records; self-hostable nodes; sovereign generated apps.

19.5 Forkability

If a future steward violates the constitutional invariants, builders should be able to fork the factory, node software, protocol schemas, public indexes, registry readers, and continuity clients.

*A protocol that cannot survive disagreement
is a product, not a protocol.*

20 The Genesis Shot

20.1 The bootstrap problem

The TOHSENO mobile app is intended to become the premium identity and signing surface for future public records. But it cannot authorize its own creation before it exists. The protocol therefore bootstraps honestly:

1. The pre-stable system uses a local or external signer.
2. The factory reaches its first stable release.
3. That stable release creates the TOHSENO mobile app through the ordinary tohseno create path.
4. The app evolves through the ordinary Shot lifecycle.
5. Its source is published.
6. Its public lineage is signed using the bootstrap signer.
7. Once the mobile signer is operational, control can rotate or extend to it.
8. \$TOHSENO is linked through the same appcoin mechanism offered to future builders.
9. The app reaches the App Store through the same lifecycle semantics used by every later Shot.

The bootstrap is part of the history, not an embarrassment to hide.

20.2 No special template

The stable factory must be capable of creating the TOHSENO mobile app without private generators or hand-written scaffolding unavailable to other builders. It may use ordinary public skills, templates, and platform capabilities. The Genesis Shot should demonstrate the system, not receive an exemption from it.

Forbidden before genesis: a mobile app target in the factory repository; a prebuilt mobile repository waiting in the wings; a privileged template that bypasses the ordinary path; a mobile-only identity shortcut; a fabricated genesis Shot ID or evidence record.

20.3 Genesis readiness

The first stable release is not ready merely because the CLI runs. It is ready when it can create and evolve the class of application required by the Genesis Shot, including native iOS foundations; secure local storage; networking; pairing; signing abstractions; subscriptions; feedback interfaces; Shot browsing; protocol integration; App Store readiness; and independent source ownership.

20.4 The recursive closure

The intended recursion is:

```

TOHSENO creates the TOHSENO app
      |
      v
the TOHSENO app holds continuity identity
      |
      v
continuity identity authorizes future TOHSENO records
      |
      v
those records preserve the lineage of future Shots
      |
      v
some of those Shots create new continuity apps

```

The system becomes capable of remembering the objects it helps bring into existence.

21 What TOHSENO is not, and the negative space

TOHSENO is not a promise that every idea deserves a market. It is a promise that a coherent idea deserves a real attempt at embodiment.

TOHSENO is not: a mandatory cloud runtime; a centralized app account; an App Store replacement; a token launcher disguised as a developer tool; an NFT minting platform; a universal user tracking system; a new blockchain; a DAO before a product; a node-reward farm; an agent-specific wrapper; a repository of everyone's private prompts; a system that owns generated source; a social network that must be joined before creating; a claim that code generation alone is continuity.

It may interoperate with many of these systems. It must not lose its identity inside them.

There is a deeper pattern here worth naming. TOHSENO defines itself as much by absence as by presence, and it treats absence as a property to be enforced, not merely intended. There is no global seed phrase — by rule. There is no hand-built mobile app — by rule, until genesis. There are no protocol fields for private prompts — by schema. There is no fabricated usage number — by brand contract. The official hostname may not be hardcoded into replaceable protocol code — by mechanical check. In the places where most systems accumulate quiet dependencies and quiet exaggerations, TOHSENO keeps tested emptiness.

Absence, here, is a tested property.

22 Constitutional invariants

The following statements define TOHSENO more deeply than any implementation.

22.1 The Shot

1. One coherent intention creates one Shot.
2. A Shot is the application-level object.
3. A Shot evolves.
4. An Evolution is not a new Shot.
5. The Shot ID remains stable across names, versions, owners, and releases.

22.2 Lifecycle

6. The canonical lifecycle is EVOLVING $\rightarrow$ PUBLISHED $\rightarrow$ APP_STORE.
7. PUBLISHED means verifiable source is available through the ecosystem.
8. APP_STORE means released through Apple.
9. A Shot may continue evolving after every lifecycle milestone.
10. Recording and lifecycle are orthogonal.

22.3 Sovereignty

11. The local factory requires no account, mobile app, wallet, token, official node, or blockchain.
12. Every generated Shot remains independently operable and ejectable from birth.
13. TOHSENO network features are additive.
14. Builders own their source.
15. The official service must not become a kill switch over local apps.

22.4 Identity

16. There is no global TOHSENO seed phrase.
17. Every human controls an independent root.
18. Generated apps never receive the root secret.
19. App-specific identities are distinct by default.
20. Cross-app linkage requires explicit human consent.

- 21. Builder identity and user identity are different roles.
- 22. Authorship history cannot be silently transferred.

22.5 Mobile

- 23. The TOHSENO mobile app is optional.
- 24. It may act as a premium identity vault, signer, feedback layer, and continuity client.
- 25. It must not be required for open-source creation or app operation.
- 26. It must be the Genesis Shot created by the first stable release.
- 27. It must not be hand-built into the pre-stable repository.

22.6 Network

- 28. A node witnesses, verifies, stores, serves, and relays.
- 29. A node is not the builder or publisher.
- 30. The official node is the first node, not the network.
- 31. Clients should be able to use compatible alternative nodes.
- 32. Nodes verify objective evidence rather than artistic worth.
- 33. Existing chains provide consensus; TOHSENO need not invent one.

22.7 Registry

- 34. Public records are append-only.
- 35. Records contain small signed facts and content hashes.
- 36. Private source and secrets do not belong onchain.
- 37. Nodes may sponsor gas without becoming authors.
- 38. Authorship records are not ordinary transferable NFTs.
- 39. Public claims must say exactly what they prove.

22.8 Appcoins

- 40. Appcoins are optional.
- 41. An appcoin is linked to a Shot; it is not the Shot.
- 42. Deployment remains launcher-agnostic.
- 43. The registry does not need to deploy tokens.

- 44. Fee routing must be explicit.
- 45. \$TOHSENO must never be required to create, own, or run a Shot.

22.9 Continuity and truth

- 46. The full loop is intention → Shot → use → feedback → next intent → Evolution.
- 47. Continuity should preserve the human relationship to the app, not merely its files.
- 48. Artificial intelligence interprets meaning; deterministic code owns composition, verification, and authority.
- 49. Claims are classified honestly: implemented, prepared, proposed, or open — and the system never blurs them.
- 50. The protocol must remain understandable by future humans and machines; old public records remain interpretable; and the system must be forkable if its steward abandons these invariants.

23 Open questions

This document fixes the ontology, not every implementation decision. The project keeps its unresolved decisions in a public list whose stated goal is its own deletion: *the asymptote of that file is empty*. At the time of writing, the following remain deliberately open:

- The path currently stops at Simulator: how should the factory accompany the human ladder to a physical device and a live listing without pretending to automate what it cannot?
- Which canonical chain should host the first registry?
- Which signature scheme best balances wallet compatibility, passkeys, and device security?
- What is the production Builder trust root, and how should competing valid histories for the same Shot ID be resolved across nodes? (Protocol version 1 has no fork-resolution rule.)
- How should source availability be proven across independent nodes?
- Which manifest storage should be default before decentralized pinning is mature?
- How should organization and co-creator control be represented?
- What constitutes strong App Store ownership verification?
- How should key recovery work without making the official mobile app custodial?
- Which feedback should be public, private, encrypted, or anonymous?
- How should a published Shot declare licensing and fork relationships?

- How should appcoin migrations and compromised coins be handled?
- Which measurable node services deserve rewards, and how does the network prevent Sybil farming?
- Should nodes stake \$TOHSENO, earn it, burn it, or use another coordination model?
- How can a user prove cross-app facts without revealing linkable identifiers?
- Which continuity capabilities belong in every generated app template?
- Can any commercial layer be added while preserving the account-free local factory?
- What exactly must the stable release demonstrate before the Genesis Shot is created?

These questions should be answered by preserving the invariants, building the smallest truthful implementation, and observing real use.

24 The wager

TOHSENO makes a wager about the future of software.

It wagers that there will not merely be more applications. There will be more **personal applications**: software specific enough to embody the taste, rituals, needs, obsessions, relationships, and strange internal logic of individual people.

It wagers that a person may carry dozens or hundreds of living Shots — a contact sheet that grows for a lifetime.

It wagers that the cost of creating a first form will continue falling, and that the scarce things will become: coherent intention; taste; attention; honest feedback; continuity; trust; distribution; public memory.

It wagers that most shots will miss, and that this is precisely why the system works: because a missed shot that exists teaches more than a perfect shot that was never taken, and because taste is developed on the contact sheet, not in the pitch deck.

It wagers that builders will want the freedom of local ownership and the benefits of a network without being forced to choose between them.

It wagers that identity can be continuous without becoming a surveillance identifier.

It wagers that an app can acquire an economic life without its token-launch mechanism becoming its ontology.

It wagers that the most important future software may begin not as a startup, but as one person saying:

I wish this existed.

TOHSENO gives that sentence a Shot.

25 Closing

A Shot begins as a circle drawn around an intention.

Inside the circle, the idea becomes executable. Then reality touches it. The app is used. It resists. It surprises. It teaches the builder what the original intention could not know. Feedback returns. A sharper intent appears. The Shot evolves.

Some Shots remain private and serve one human well. Some are published and invite others inside. Some reach the App Store. Some become communities. Some acquire appcoins. Some are forgotten. Some survive their creators.

The protocol does not decide which path deserves to happen. Its responsibility is simpler and harder:

- keep the factory open;
- keep the artifact sovereign;
- keep identity in the human's hands;
- keep public claims legible;
- keep the network replaceable;
- keep history intact;
- and preserve the connection between an intention and everything that grows downstream from it.

The first stable release will create the TOHSENO mobile app. That app will become the first canonical Shot of the system that created it. From there, the circle opens.

And when it does, the invitation will be the same one printed at the end of every build, the one this whole system exists to make easy to accept:

Take another one.



Colophon

This document is Version 0.2 of the TOHSENO Genesis Whitepaper. It supersedes the preliminary draft of July 25, 2026, and was written against factory release 0.5.0, before the first stable release and before the Genesis Shot exists.

It is versioned deliberately. Version 1.0 of this document is reserved: it will be finalized alongside the first stable release and signed with the bootstrap builder identity, so that the constitution and the system it describes reach stability together. The loom must be complete before it weaves its first flag.

The document is typeset in EB Garamond and IBM Plex Mono, in the darkroom palette of the TOHSENO factory: near-black ink, silver-halide grey, and one hot signal color. Page numbers are exposure numbers. If you are reading this from a permanent content-addressed store, then this file has become what it describes: a small signed fact, preserved beyond any single server, waiting for whoever finds it.

TOHSENO / GENESIS WHITEPAPER / V0.2 / JULY 2026
GIVE EVERY IDEA A SHOT. MOST SHOTS MISS. THE PROTOTYPE IS THE PAYOFF.
END OF ROLL 01